

Exercise 3: Sorting Customer Orders

1. Aim

To implement **Bubble Sort** and **Quick Sort** algorithms in Java for sorting customer orders based on their total price and compare their performance.

2. Problem Understanding

In an e-commerce platform, customer orders often need to be sorted based on their total price. This helps businesses prioritize high-value orders for processing and delivery.

Sorting algorithms arrange data in a specific order, making it easier to analyze and process information efficiently.

3. Sorting Algorithms

Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest element moves to the end of the array.

Characteristics:

- Simple to understand and implement.
- Suitable only for small datasets.
- Inefficient for large datasets.

Insertion Sort

Insertion Sort builds the sorted array one element at a time by inserting each element into its correct position.

Characteristics:

- Efficient for small or nearly sorted datasets.
- Better than Bubble Sort for partially sorted data.

Quick Sort

Quick Sort uses the Divide and Conquer technique. It selects a pivot element, partitions the array into smaller and larger elements, and recursively sorts the partitions.

Characteristics:

- Very efficient for large datasets.
- One of the fastest comparison-based sorting algorithms.

Merge Sort

Merge Sort divides the array into two halves, recursively sorts each half, and then merges them into a sorted array.

Characteristics:

- Stable sorting algorithm.
- Guaranteed $O(n \log n)$ performance.
- Requires additional memory.

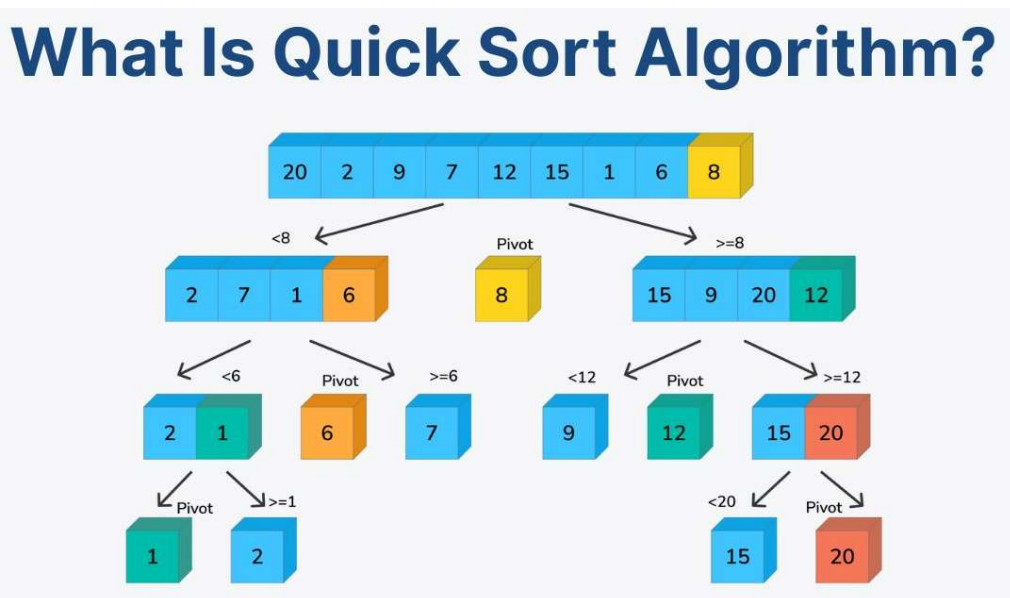
4. Algorithm

Bubble Sort

1. Compare adjacent orders.
2. Swap if the first order has a greater total price.
3. Repeat until the array is sorted.

Quick Sort

1. Select the last element as the pivot.
2. Partition the array around the pivot.
3. Recursively sort the left and right partitions.
4. Repeat until the entire array is sorted.



5. Program/Output:

```
J Exercise3.java > ...
1  class Order{
2      int orderId;
3      String customerName;
4      double totalPrice;
5
6      Order(int orderId,String customerName,double totalPrice){
7          this.orderId = orderId;
8          this.customerName = customerName;
9          this.totalPrice = totalPrice;
10     }
11
12     public String toString(){
13         return "Order Id: "+orderId+
14             ", Customer: "+customerName+
15             ", Total Price: Rs."+totalPrice;
16     }
17 }
18 public class Exercise3 {
19     public static void bubbleSort(Order[] orders){
20         int n = orders.length;
21
22         for(int i=0;i<n-1;i++){
23             {
24                 for(int j=0;j<n-1-i;j++){
25                     if(orders[j].totalPrice>orders[j+1].totalPrice){
26                         Order temp = orders[j];
27                         orders[j] = orders[j+1];
28                         orders[j+1] = temp;
29                     }
30                 }
31             }
32         }
33     public static void quickSort(Order[] orders,int low,int high){
34         if(low<high){
35             int pivotIndex = partition(orders, low, high);
36
37             quickSort(orders, low, pivotIndex-1);
38             quickSort(orders, pivotIndex+1, high);
39         }
40     }
41     public static int partition(Order[] orders,int low,int high){
42         double pivot = orders[high].totalPrice;
43         int i= low -1;
44         for(int j=low;j<high;j++){
45             if(orders[j].totalPrice<=pivot){
46                 i++;
47                 Order temp = orders[i];
48                 orders[i] = orders[j];
49                 orders[j] = temp;
50             }
51         }
52         Order temp = orders[i+1];
53         orders[i+1] = orders[high];
54         orders[high] = temp;
55         return i+1;
56     }
57     public static void displayOrders(Order[] orders){
58         for(Order ord: orders){
59             System.out.println(ord);
60         }
61     }
}
```

```

62  public static void main(String[] args) {
63      Order[] orders = {
64          new Order(orderId: 100, customerName: "Krishna", totalPrice: 5000),
65          new Order(orderId: 101, customerName: "Vishwa", totalPrice: 12000),
66          new Order(orderId: 102, customerName: "Anjali", totalPrice: 7000),
67          new Order(orderId: 103, customerName: "Priya", totalPrice: 4000),
68          new Order(orderId: 104, customerName: "Arun", totalPrice: 4500)
69      };
70      System.out.println(x: "Original Orders");
71      displayOrders(orders);
72
73      bubbleSort(orders);
74
75      System.out.println(x: "\nAfter Bubble Sort");
76      displayOrders(orders);
77
78      Order[] orders2 = {
79          new Order(orderId: 100, customerName: "Krishna", totalPrice: 5000),
80          new Order(orderId: 101, customerName: "Vishwa", totalPrice: 12000),
81          new Order(orderId: 102, customerName: "Anjali", totalPrice: 7000),
82          new Order(orderId: 103, customerName: "Priya", totalPrice: 4000),
83          new Order(orderId: 104, customerName: "Arun", totalPrice: 4500)
84      };
85
86      quickSort(orders2, low: 0, orders2.length-1);
87
88      System.out.println(x: "\nAfter Quick Sort:");
89      displayOrders(orders2);
90  }
91  }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Original Orders

Order Id: 100, Customer: Krishna, Total Price: Rs.5000.0
 Order Id: 101, Customer: Vishwa, Total Price: Rs.12000.0
 Order Id: 102, Customer: Anjali, Total Price: Rs.7000.0
 Order Id: 103, Customer: Priya, Total Price: Rs.4000.0
 Order Id: 104, Customer: Arun, Total Price: Rs.4500.0

After Bubble Sort

Order Id: 103, Customer: Priya, Total Price: Rs.4000.0
 Order Id: 104, Customer: Arun, Total Price: Rs.4500.0
 Order Id: 100, Customer: Krishna, Total Price: Rs.5000.0
 Order Id: 102, Customer: Anjali, Total Price: Rs.7000.0
 Order Id: 101, Customer: Vishwa, Total Price: Rs.12000.0

After Quick Sort:

Order Id: 103, Customer: Priya, Total Price: Rs.4000.0
 Order Id: 104, Customer: Arun, Total Price: Rs.4500.0
 Order Id: 100, Customer: Krishna, Total Price: Rs.5000.0
 Order Id: 102, Customer: Anjali, Total Price: Rs.7000.0
 Order Id: 101, Customer: Vishwa, Total Price: Rs.12000.0

PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\Algorithm>

6. Time Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

7. Comparison

Bubble Sort	Quick Sort
Simple to implement	More efficient for large datasets
Compares adjacent elements	Uses Divide and Conquer
$O(n^2)$ average time	$O(n \log n)$ average time
Slow for large data	Faster and widely used

8. Result

Thus, the **Bubble Sort** and **Quick Sort** algorithms were successfully implemented to sort customer orders based on their total price. Bubble Sort is simple but inefficient for large datasets, whereas Quick Sort provides significantly better performance and is therefore the preferred choice for sorting customer orders in an e-commerce platform.